SMT over finite fields, ring-theoretically

companion note to *Training for Constraining*, Devcon 8

1. Soundness is a question about one ideal

A circuit over $\mathbb{F}_p$ is a list of polynomials $C = \{f_1, \ldots, f_m\} \subset \mathbb{F}_p[x, \overline{w}]$ in the public witness x and the extended witness $\overline{w}$: the secret witness w plus intermediates and hints. The accepted assignments are the $\mathbb{F}_p$-rational points of $V(f_1, \ldots, f_m)$. Soundness says every accepted assignment satisfies the specification R, and the solver refutes its negation (slide 39):

$$\forall(\overline{w}, x):\ C(\overline{w}, x) = 0\ \Rightarrow\ R(w, x) \qquad \text{fails iff} \qquad \exists(\overline{w}, x) \in \mathbb{F}_p^n:\ C(\overline{w}, x) = 0\ \wedge\ \neg R(w, x).$$

Encode $\neg R$ as polynomials in fresh variables t (§2) and the whole query is one ideal

$$\boxed{I = \langle f_1, \ldots, f_m,\ \text{encoding of } \neg R \rangle\ \subset\ \mathbb{F}_p[x, \overline{w}, t]}$$

The circuit is sound exactly when $V(I)$ has no point with all coordinates in $\mathbb{F}_p$. The algebraic test is whether $1 \in I$ (§3).

2. Encoding $\neg R$

A specification is a Boolean combination of polynomial equalities, and over a field each connective has an exact polynomial form:

Logic	Polynomial	Why it is exact over a field
$g = 0\ \wedge\ h = 0$	generators g and h	sum of ideals
$g = 0\ \vee\ h = 0$	$g\,h$	no zero divisors
$g \neq 0$	$t\,g - 1$	**Rabinowitsch**: $t = g^{-1}$ exists iff $g \neq 0$ (localisation at g)
$g \neq 0\ \wedge\ h \neq 0$	$t\,gh - 1$	a product of nonzero elements is nonzero
$g \neq 0\ \vee\ h \neq 0$	$t_1 g + t_2 h - 1$	solvable iff $(g, h) \neq (0, 0)$: take $t_1 = g^{-1}$, $t_2 = 0$ or vice versa

Two cases cover most circuits. If R is a conjunction $g_1 = \cdots = g_k = 0$ ("the output equals the intended function"), then $\neg R$ is the single polynomial $t_1 g_1 + \cdots + t_k g_k - 1$. The selector's R is a disjunction, $z = x \vee z = y$, so $\neg R$ is $z \neq x \wedge z \neq y$: again one polynomial, $t\,(z - x)(z - y) - 1$.

3. The test runs one way

$$1 \in I\ \Longrightarrow\ V(I) = \varnothing\ \Longrightarrow\ C \wedge \neg R \text{ has no solution in } \mathbb{F}_p\ \Longrightarrow\ \text{sound.}$$

The first arrow is immediate: evaluate $1 = \sum_i h_i f_i$ at a common zero to get $1 = 0$. The weak Nullstellensatz reverses it over the algebraic closure $\overline{\mathbb{F}}_p$ only, because $V(I)$ may have points that are not $\mathbb{F}_p$-rational. The smallest example is $x^2 + 1 \in \mathbb{F}_3[x]$: no root in $\mathbb{F}_3$, yet $\langle x^2 + 1 \rangle$ is a proper ideal, so $1 \notin I$. Hence $1 \notin I$ **does not mean "bug"**, and there are two ways to close the gap.

Search for an $\mathbb{F}_p$-point of $V(I)$: cvc5's model-finding phase (§4).

Add the field equations $v^p - v$ for every variable v: then V contains only $\mathbb{F}_p$-points, and $1 \in I + \langle v^p - v \rangle$ holds exactly when there is no $\mathbb{F}_p$ solution. In the example, $1 = (x^2 + 1)^2 - x\,(x^3 - x)$ over $\mathbb{F}_3$. At $p \approx 2^{254}$ these generators have astronomical degree, which is why solvers search instead.

Logic / solver	Commutative algebra
conjunction of constraints	ideal generated by the polynomials
UNSAT over $\overline{\mathbb{F}}_p$	$1 \in I$ (weak Nullstellensatz: $V(I) = \varnothing\ \Longleftrightarrow\ 1 \in I$)
UNSAT over $\mathbb{F}_p$ exactly	$1 \in I + \langle v^p - v \rangle$ (field equations cut V down to $\mathbb{F}_p^n$)
SAT with model	an $\mathbb{F}_p$-point of $V(I)$; the model is its coordinates
unsat core	which generators occur in the certificate $1 = \sum_i h_i f_i$

Every conclusion is about the encoded model. A certificate proves that this I has no $\mathbb{F}_p$-point. That I is the circuit the prover actually runs, and that the encoded R is the intended one, sit outside the algebra.

4. What cvc5 actually computes (no sampling anywhere)

Refutation. It computes a Gröbner basis of I (Buchberger/F4) for a monomial order and reduces 1. The reduced basis is $\{1\}$ iff $1 \in I$, and the computation produces an explicit **Nullstellensatz certificate** $1 = \sum_i h_i f_i$: a finite identity that is a proof of emptiness for all p^n candidate points at once. If the h_i have integer coefficients the identity holds in every commutative ring, which is why the same UNSAT is returned over $\mathbb{F}_{11}$ and over the 254-bit BN254 field in the same few milliseconds.

Model finding. If the basis is not $\{1\}$, $V(I) \neq \varnothing$ over $\overline{\mathbb{F}}_p$ but an $\mathbb{F}_p$-point still has to be found. Zero-dimensional case: a lexicographic basis is triangular, so solve the univariate element by exact root finding in $\mathbb{F}_p$ (gcd with $x^p - x$, Cantor–Zassenhaus) and back-substitute. Positive-dimensional case: fix a free variable, recompute, recurse, backtrack. Each step is exact elimination; the returned model is re-checked against every constraint.

Three answers. cvc5 omits the field equations of §3. So UNSAT is a proof (even over $\overline{\mathbb{F}}_p$), SAT comes with a checkable witness, and a third answer, UNKNOWN, exists when the point search gives up or times out. A fuzzer can only ever say SAT; it has no object corresponding to the certificate.

How big the basis gets. For n variables of degree d, Gröbner basis degrees are bounded by $d^{2^{O(n)}}$ (Dubé), and ideal membership needs doubly exponential degrees in the worst case (Mayr–Meyer). With finitely many solutions over $\overline{\mathbb{F}}_p$ the bounds drop to single exponential, $d^{O(n)}$. Over $\mathbb{F}_p$ there is one relief: coefficients never grow, since each stays a $\log_2 p$-bit field element, so the

blow-up is only in the number and degree of terms. Structure matters more than size: circomlib's `Num2Bits` at $b = 32$ did not finish in a week (§8). Every practical method shrinks what one basis is computed over: split bases (§9), one gadget at a time, or one slice of a circuit at a time.

5. Worked example: the selector

Constraint $f_1 = z - sx - (1-s)y$ and repair $f_2 = s(s-1)$ (slide 21). The negated specification $z \notin \{x, y\}$ is the single polynomial $g = t\,(z-x)(z-y) - 1$ of §2. Reducing modulo f_1,

$$(z-x)(z-y) \;\equiv\; (1-s)(y-x)\cdot s(x-y) \;=\; s(s-1)(x-y)^2 \pmod{f_1},$$

and f_2 kills the right-hand side. Writing $h = z + sx + (1-s)y - x - y$ for the cofactor of f_1, the identity (verified by expansion)

$$\boxed{\;1 \;=\; t\left[h\,f_1 + (x-y)^2 f_2\right] \;-\; g\;}$$

is the certificate: $1 \in I$, so the repaired selector is sound in every field. With two Rabinowitsch variables, $g_1 = t_1(z-x) - 1$ and $g_2 = t_2(z-y) - 1$, it reads $1 = t_1 t_2[h\,f_1 + (x-y)^2 f_2] - (g_1 g_2 + g_1 + g_2)$. Delete f_2 and $\langle f_1, g\rangle$ has the $\mathbb{F}_{11}$-point $(x, y, z, s) = (3, 7, 5, 6)$ with $t = (2\cdot(-2))^{-1} = 8$: cvc5 answers SAT and prints it. The Lean theorem `selector_sound` is the same argument in two cases via `mul_eq_zero`.

Where the field axiom enters. Over $\mathbb{Z}/6$, $s = 3$ satisfies $s(s-1) = 0$ and gives $z = 3x + 4y = 4 \notin \{x, y\}$ for $(x, y) = (0, 1)$, yet the certificate is a valid ring identity. No contradiction: $(z-x)(z-y) = 4 \cdot 3 = 0$ in $\mathbb{Z}/6$, so $g = -1$ and the encoding reports "$\neg R$ fails" although $z \notin \{x, y\}$. Both readings in §2, "$\neq 0$" as "is a unit" and "$\wedge$" as "the product is nonzero", hold exactly in a field; that coincidence is what "sound over every field" rests on.

6. Public and private wires play different roles

The variables of I are not interchangeable. The verifier fixes the public x: it *is* the claim. The prover chooses $\overline{w}$, possibly adversarially. A free private value is harmless if it cannot reach x, like the hint of slide 18; a public value not tied to the computation is the bug, as in Example 0. The labels decide which ideal answers which question.

Statement level: eliminate the private wires. With a lexicographic order putting $\overline{w}$ above x, a Gröbner basis of $\langle C\rangle + \langle v^p - v\rangle$ contains a basis of the elimination ideal $I_x = \big(\langle C\rangle + \langle v^p - v\rangle\big) \cap \mathbb{F}_p[x]$. Its zeros are exactly the public statements the circuit can prove. For the circuit of slide 9 over $\mathbb{F}_{11}$:

Constraints	$I_x \subset \mathbb{F}_{11}[x_0]$	public values accepted
C_0, C_1, C_2 (Example 0: C_3 dropped)	$\langle x_0^{11} - x_0\rangle$	all of $\mathbb{F}_{11}$, including the false 7
$C_0, \ldots, C_3$	$\langle (x_0-3)(x_0-5)(x_0-6)(x_0-8)(x_0-9)(x_0-10)\rangle$	$\{3, 5, 6, 8, 9, 10\} = \{w_0^4 + 5\}$

Statement-level soundness is the containment $V_{\mathbb{F}_p}(I_x) \subseteq \{x : \exists w\ R(w, x)\}$.

Instance level: substitute the public values. Plug numbers in for x; only private wires remain variables, and the question is whether *this* claim has a witness. Pinning the selector to $(3, 7, 5)$ is this case.

Determinism: duplicate the private side. Tools like Picus share the inputs x, copy every other wire, and ask whether the outputs can differ: $C(\overline{w}, x, y) = 0 \wedge C(\overline{w}', x, y') = 0 \wedge \sum_i t_i(y_i - y_i') - 1 = 0$, the disjunction row of §2. This is the query split Gröbner bases attack (§9).

Elimination needs the field equations to see only $\mathbb{F}_p$-points, so at $p \approx 2^{254}$ it is the conceptual picture; solvers work instance by instance or with the determinism query.

7. cvc5 versus Zirgen: different layers of the stack

cvc5 is a general SMT solver (Stanford/Iowa). Its finite-field theory `QF_FF` (Ozdemir, Kremer, Tinelli, Barrett, CAV 2023; split Gröbner bases, CAV 2024) decides polynomial systems over $\mathbb{F}_p$ as in §4. Input: constraints in SMT-LIB. Output: SAT+model, UNSAT, or UNKNOWN. It *analyses* circuits and is the backend of Picus; it never produces one.

Zirgen is RISC Zero's circuit DSL and MLIR-based compiler. It *produces* circuits: AIR constraints plus witness generation, emitted as Rust/C++/CUDA/Metal prover and verifier code for the RISC Zero zkVM. It decides nothing. Verification reaches it from outside: Nethermind's Lean model of Zirgen's MLIR with a verification-condition generator, and Veridise's zirgen-to-LLZK frontend, through which Picus (hence cvc5) and zkLean become available.

In the language of this note: Zirgen is where the ideal I gets written down; cvc5 is one of the procedures that decides whether I has a rational point. Compiler versus decision procedure, not competitors.

References. Cox–Little–O'Shea, *Ideals, Varieties, and Algorithms* (Nullstellensatz, Gröbner bases, elimination). Mayr and Meyer, Adv. Math. 46 (1982); Dubé, SIAM J. Comput. 19 (1990) (degree bounds). Ozdemir et al., *Satisfiability Modulo Finite Fields*, CAV 2023 (ePrint 2023/091). Ozdemir et al., *Split Gröbner Bases for Satisfiability Modulo Finite Fields*, CAV 2024 (ePrint 2024/572). Pailoor et al., *Automated Detection of Under-Constrained Circuits in ZK Proofs*, PLDI 2023 (Picus). `https://github.com/risc0/zirgen`; `https://github.com/NethermindEth/risczero-fv`; `https://github.com/project-llzk/llzk-lib`.

Why split Gröbner bases, where they stop, and when to switch to Lean

part 2 of the companion note; all figures from Ozdemir et al., CAV 2024

8. The bitsum wall

Take the simplest range check there is, for $p \geq 2^b$:

$$\bigwedge_{i=1}^{b} X_i(1 - X_i) = 0, \qquad X_1 + 2X_2 + \cdots + 2^{b-1}X_b = 0, \qquad X_b Z = 1.$$

A human refutes it in one line: the bits are Boolean, so the bitsum forces every $X_i = 0$, contradicting $X_b Z = 1$. A full Gröbner basis for this system takes time *exponential in* b. The 2023 cvc5 solver could not decide determinism of circomlib's `Num2Bits` at $b = 32$ "even in a week". The diagnosis (their §3.2) is sharp: the bitsum alone is already a Gröbner basis, the bit constraints alone are already a Gröbner basis, and it is *their combination* that blows up. Translating to bit-vectors is a dead end because the surrounding equations are genuinely field equations. And the pattern is not exotic: 98% of public Circom projects use bitsums, and 64% of the variables in circomlib's SHA-2 sit inside one. This is slide 24 of the workshop at industrial scale.

9. The fix: several small bases plus one non-algebraic fact

A **split Gröbner basis** for I is a tuple $(B_1, \ldots, B_k)$ of Gröbner bases with $\langle B_1 \cup \cdots \cup B_k \rangle = I$. Each B_i answers ideal membership for its own subsystem only; in exchange it may be cheap. The procedure Split refines the tuple by passing polynomials between bases, and a hook extraProp may inject any polynomial provably in I by *other* means. The concrete instance BitSplit uses $k = 2$: a *sparse* basis holding everything except bitsums, and a *linear* basis holding all linear polynomials including bitsums, so that a bitsum and its bit constraints never meet in one Buchberger run. Its single propagation rule is **congruence of binary representation**: if $Y - \sum 2^{i-1}X_i$ and $Y' - \sum 2^{i-1}X'_i$ are present, all bits are known Boolean, $m < \log_2 |\mathbb{F}|$, and $Y - Y' \in I$, then every $X_i - X'_i \in I$. For `Num2Bits` determinism that is the whole proof: with $f = X'_j - X_j$ propagated and $g = (X'_j - X_j)Z - 1$ the disequation, $1 = fZ - g \in I$. Soundness and termination are proved (their Thm. 3), and the search phase is complete in principle. Numbers at a 300 s limit over 1140 benchmarks:

	all solved	`QED2` (100)	CirC determinism (640)	CirC soundness (100)
Monolithic (2023)	475	13	191	38
BitSplit (2024)	969	100	582	59
BitSplit without the bitsum propagator	344	0	131	34

10. What it does not fix

1. **The worst case is unchanged.** "In the worst case, BitSplit builds a GB for the full system"; complexity stays doubly exponential. The gain is heuristic, and the third row above shows it is almost entirely the propagator.
2. **The propagator matches a syntactic pattern.** It fires only on $Y - \sum 2^{i-1}X_i$ with $m < \log_2 |\mathbb{F}|$ and bits already known Boolean. Signed-digit or base-2^k limb decompositions, bitsums rewritten by earlier algebra, and any decomposition with $2^m > p$ (the slide-24 pitfall, where representation is not unique) receive no help and fall back to raw Gröbner computation.
3. **Integer semantics are out of reach.** The authors' own account of what remains: "BitSplit struggles only with division and remainder; verifying their determinism would require understanding that integer division is deterministic, as encoded in field constraints." That is precisely the quotient-and-range argument of slides 25–26 and of foreign-field gadgets like fake GLV.
4. **Determinism is not soundness.** The breakthrough is on determinism queries, $\Phi(X, Y) \wedge \Phi(X, Y') \wedge Y \neq Y'$, whose specification writes itself. Queries against a real specification R (CirC soundness, 59/100) remain hard, and the selector of §5 is sound for a reason determinism alone cannot express.
5. **One query per bitwidth and per field.** Each benchmark fixes b and p. Nothing is learned about $b + 1$.
6. **Incomplete internal propagation and thin CDCL(T) integration**, both acknowledged: linear-to-sparse propagation is incomplete (their Example 1, and completing it did not help), and the solver "constructs no theory lemmas".

11. When to rely on a Lean proof instead

A Lean proof is not a search, so it has no unknown: a human supplies the argument and the kernel checks it, and the cost is human time rather than solver time. That cost is worth paying exactly where the list above says the solver stops. **Parametric statements.** "Binary representation is unique whenever $2^b \leq p$" is one theorem for all b and all p, a short induction; the solver needs one query per (b, p) and times out past $b \approx 32$ for the hard operators. **Integer semantics.** The argument "$xy - z - 3k \in \{-5, \ldots, 4\}$ and the only multiple of 11 there is 0" is a lift from $\mathbb{F}_p$ to $\mathbb{Z}$ followed by linear arithmetic; in Lean that is `omega` after `ZMod.val` bookkeeping, and it scales to 254-bit moduli and multi-limb representations unchanged. This is the division-and-remainder gap, closed. **Specifications, not just determinism.** `selector_sound` and `scalar_cancellation` in the repository state $C \Rightarrow R$ against the intended relation, over an arbitrary field and an arbitrary module, which no determinism query can. **Group-theoretic hypotheses.** Prime-order subgroup membership and $v \not\equiv 0 \pmod{\ell}$ are hypotheses about a module over $\mathbb{F}_\ell$, not polynomial identities in $\mathbb{F}_r$; they are natural in Lean and have no SMT encoding of practical size.

Working rule. Run the push-button tool first: sat is an exploit, unsat closes a fixed-size gadget. Escalate to Lean when the answer is unknown, when the property is parametric in b or p, when the proof needs integers or groups rather than field identities, or when the property is soundness against a specification rather than determinism. The caveat of §3 still applies: the theorem is about the model, so the circuit-to-model step (LLZK, zkLean) is where the remaining trust sits.